

实验五 循环程序设计（二）while 语句

一、 实验目的：

1. 熟练掌握 while 语句和 do...while 语句实现条件循环结构程序设计；
2. 熟练掌握 break 和 continue 语句在循环中的使用；
3. 熟练掌握循环嵌套程序设计。

二、 实验要求：

1. 用 C 语言实现源程序的编写；
2. 源程序应书写规范，适当添加注释；
3. 实验课前完成实验内容源程序的初稿；实验课对源程序进行编辑、调试、运行；实验课后尽快完成实验报告，并按规定时间上交实验报告。

三、 实验源代码、结果及流程图：

1. 猜数字游戏：先输入 2 个不超过 100 的正整数，分别是被猜数 m_n 和允许猜测的最大次数 n，再输入你所猜的数 y_n，与被猜数 m_n 进行比较，若相等，显示猜中；若不等，显示与被猜数的大小关系，最多允许猜 n 次。如果 1 次就被猜出该数，提示“Bingo!”；如果 3 次以内猜到该数，则提示“LuckyYou!”；如果超过 3 次但不超过 n 次猜到该数，则提示“GoodGuess!”；如果超过 n 次都没有猜到，则提示“GameOver!”；如果在到达 n 次之前，用户输入了一个负数，也提示“GameOver!”，并结束程序。试编写相应程序。

```
#include <stdio.h>

//example:4 10
int main(int argc, char *argv[]) {
    int m, n;
    scanf("%d %d", &m, &n);
    int guess ;
    for(int i = 1; i <= n; i++){
        scanf("%d", &guess);
        if(guess < 0){
            printf("GameOver!");
            break;
        }
        else if(guess == m && i == 1){
            printf("Bingo!");
            break;
        }
        else if(guess == m && i <= 3){
```

```

        printf("LuckyYou!");
        break;
    }
    else if(guess == m && i > 3 && i <= n) {
        printf("GoodGuess!");
        break;
    }
    else if(guess != m) {

        printf("Please try again!\n");
        continue;
    }
    else{
        printf("GameOver!");
        break;
    }
}
}

```

测试结果:

```

4 10
2
Please try again!
5
Please try again!
4
LuckyYou!

```

2. 猴子吃桃问题。一只猴子第一天摘下若干个桃子，当即吃下一半后，又多吃了一个；第二天早上又将剩下的桃子吃了一半后，又多吃了一个。以后每天早上都吃了前一天剩下的一半加一个。到第 n 天早上只剩下一个桃子了。问第一天一共摘了多少个桃子？试编写相应程序。

```

#include <stdio.h>

int peaches(int n){ //吃了  $n$  天
    if(n == 1)
        return 1;
    //按照递推公式递归即可
}

```

```

        return 2*(peaches(n-1) + 1);
    }
    // exampe: 10
    // expected: 1534
    int main(int argc, char *argv[]) {
        int n;
        printf("Please input total days >> ");
        scanf("%d", &n); //第n天只剩下一个桃子了

        printf("Peaches picked are >> ");
        printf("%d", peaches(n));
    }

```

测试结果:

```

Please input total days >> 10
Peaches picked are >> 1534

```

3. 求 1000 以内的所有完全数。（说明：一个数如果恰好等于它的因子之和（除自身外），则称该数为完全数，例如：6=1+2+3，6 为完全数）。试编写相应程序。

```

#include <stdio.h>

int main(int argc, char *argv[]) {
    printf("1000 以内的完全数如下:\n");
    for(int i = 6; i <= 1000; i++){
        int sum = 1;
        for(int j = 2; j < i; j++){ //求所有因数的和（不包括
自身）
            if(i % j == 0)
                sum += j;
        }
        if(i == sum)
            printf("%d ", i);
    }
}

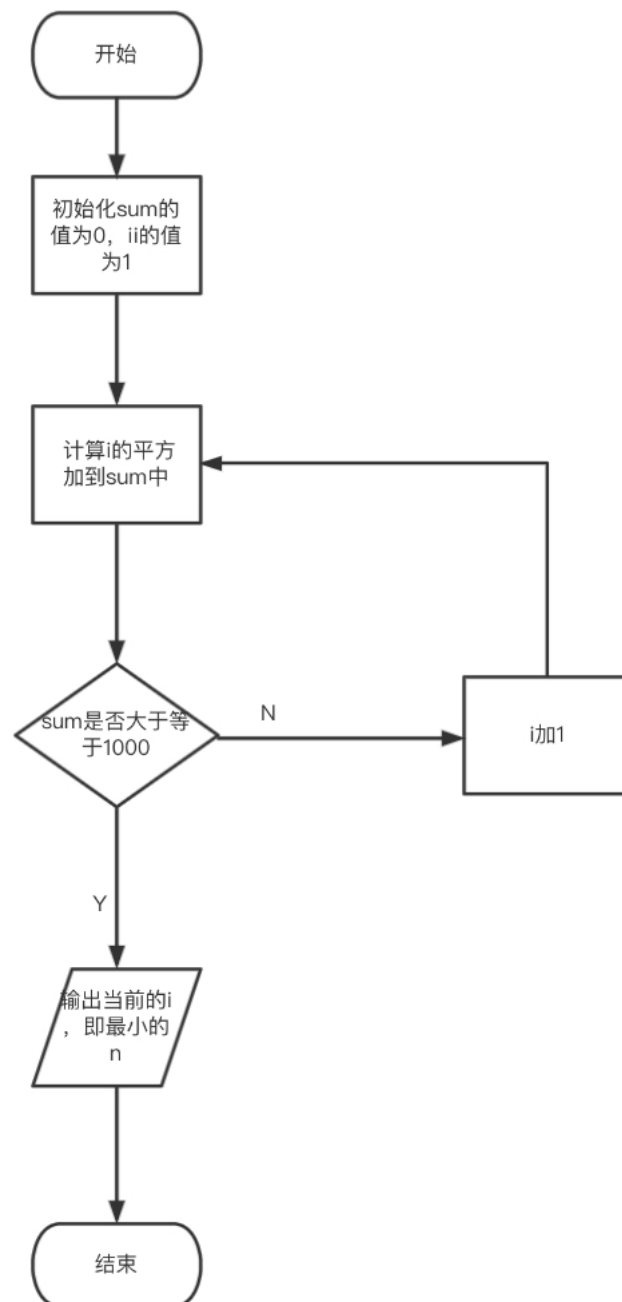
```

测试结果:

1000以内的完全数如下：
6 28 496

4. 求最小的 n 。求满足表达式 $1^2+2^2+3^2+\dots+n^2>1000$ 的最小 n 。试编写相应程序（本题要求在实验报告中画出流程图）。

程序流程图：



```
#include <stdio.h>
```

```

#include <math.h>

int main(int argc, char *argv[]) {
    int sum = 0;
    for(int i = 1; i <= 30; i++) { //枚举法的优化 (减少不必要的尝试)
        sum += pow(i, 2);
        if(sum >= 1000) {
            printf("The min n is : %d\n", i);
            break;
        }
    }
}

```

测试结果:

The min n is : 14

5. 展开式求和。输入一个实数 x , 计算并输出下式的值(计算结果保留 4 位小数), 直到最后一项的绝对值小于 0.00001。要求定义和调用函数 $\text{fact}(n)$ 计算 n 的阶乘, 可以调用 $\text{pow}()$ 函数求幂。试编写相应程序。

$$s = x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \cdots = e^x - 1$$

```

#include <stdio.h>
#include <math.h>
#include <stdlib.h>
//递归求函数阶乘
int fact(int n){
    if(n == 1)
        return 1;
    return n * fact(n - 1);
}

int main(int argc, char *argv[]) {
    double x;
    scanf("%lf", &x); //近似 x 的值
    double approximate = 0;
    for(int i = 1; i <= 10000; i++) { //自然指数泰勒公式的近似
        if(abs((double) (pow(x, i) / fact(i)) < 0.00001)) //近
似的精度
            break;
    }
}

```

```
        approximate += (double) (pow(x, i) / fact(i));  
    }  
    printf("Approximate value is : %.4lf", approximate);  
}
```

测试结果:

4

Approximate value is : 56.5065

四、 实验小结:

1. 了解了 while 语句和 do...while 语句，以及在实际开发中的对应应用场景。
2. 在解决实际问题时，优化循环结构的时候，要利用好 break 和 continue 语句，已经求出答案时要及时 break，不符合题目条件的要及时 continue
3. 联系实际，对于问题要求，要熟练地使用循环嵌套，比如很多经典的排序算法，冒泡、快排、堆排序等。

五、 实验成绩: